

대용량 통계 조회 성능 개선

㈜이씨오 | 책임음 개선 사업 · 2024 - 2025

담당 범위 · 통계 테이블 설계 · 집계 및 조회 쿼리 개발 · 인덱스 설계와 튜닝

약 30분이 걸리던 통계 조회를 100ms 이하로 단축했습니다. 통계 테이블 설계부터 배치 집계, 인덱스와 조회 쿼리 튜닝까지 담당했습니다.

문제와 접근

통계 조회에 약 30분이 소요되어 이용자가 결과를 바로 확인하기 어려웠습니다. 대규모 대출 데이터를 처리하면서 회원별 독서량과 전체 대비 순위를 제공해야 했습니다.

실행계획을 분석해 테이블 구조와 인덱스, 조회 쿼리를 개선했습니다. 대출현황·상호대차·대출이력은 배치에서 집계하고, 화면은 집계된 통계 테이블을 조회하도록 구성했습니다.

구현 흐름

대출현황·상호대차·이력 → 기간·회원 기준 집계 → 통계 테이블 MERGE → 연도별 순위·평균 조회

설계 판단

쟁점	구현한 선택	의미와 고려사항
집계와 조회 분리	대출 데이터는 배치에서 집계하고 화면은 통계 테이블 조회	원천 데이터 집계와 화면 조회의 처리 분리
집계 기준	회원 키와 연도를 기준으로 통계 갱신·삽입	회원별 연간 대출량을 동일한 기준으로 관리
순위 계산 순서	전체 회원의 순위를 계산한 뒤 대상 회원 조회	전체 대비 순위와 동점자의 동일 순위 유지
성능 튜닝	실행계획 분석 후 테이블·인덱스·조회 쿼리 개선	통계 조회 응답 시간 단축

결과

통계 조회 지연을 해결하기 위해 테이블 구조·인덱스·쿼리를 개선했고, 조회 시간을 약 30분에서 100ms 이하로 단축했습니다.

기술 · Oracle · Tiberio, 배치 집계, 실행계획 분석, RANK

전체 순위와 개인 통계 조회

연도별 통계에서 전체 회원의 순위와 평균을 계산한 뒤, 조회할 회원의 결과를 반환합니다.

전체 모집단 계산 후 개인 결과 선택

```
WITH ranked AS (  
  SELECT member_id, loan_count,  
         RANK() OVER (  
           ORDER BY loan_count DESC  
         ) AS member_rank  
  FROM annual_member_stats  
  WHERE stat_year = :target_year  
, totals AS (  
  SELECT COUNT(*) AS population,  
         AVG(loan_count) AS average_count  
  FROM ranked  
)  
SELECT r.loan_count, ROUND(t.average_count, 2),  
       ROUND(NVL(r.member_rank * 100 /  
                 NULLIF(t.population, 0), 0), 5) AS rate  
FROM ranked r CROSS JOIN totals t  
WHERE r.member_id = :member_id;
```

- 회원 조건을 순위 계산 뒤에 적용해 전체 회원 대비 순위를 유지합니다.
- RANK로 동점 회원에게 같은 순위를 부여하고, NULLIF로 0 나눗셈을 처리합니다.

핵심 로직 예시 · 업무 식별자는 변경했습니다.

지역별 회원수 수집 자동화

2025.11 | 스케줄러·20개 단위 작업 분할·지역별 통계 수집 및 저장 서비스 개발

담당자가 수동으로 하던 200개 이상 지역의 회원수 수집을 스케줄러 기반으로 자동화했습니다. 지역 목록을 20개씩 나누고 그룹 안에서 병렬 처리해, 전체 동시 호출의 부하와 순차 호출의 대기 누적을 함께 고려한 수집 구조를 구현했습니다.

도입 전 · 담당자의 수동 수집 업무

담당자가 회원수 수집 업무를 수동으로 수행했습니다. 반복되는 수집 작업에 담당자의 개입이 필요했습니다.

업무 시작과 반복에 담당자 개입

담당자

수집 업무 수행



수동 작업

지역별 회원수

수집 결과 확보



다음 수집 시

담당자 재개입

수집 업무 반복

도입 후 · 스케줄러 + 그룹 내 병렬 처리

현재 그룹의 지역 작업을 실행 스레드에 분배합니다. 실행 순서는 고정하지 않고, 그룹 내 모든 작업이 반환되면 다음 그룹으로 진행합니다.

스케줄러 실행 → 지역 목록을 20개씩 분할

그룹 1 · 지역 01-20

최대 20개 지역 작업



런타임이 실행 스레드에 작업 분배

실행 레인 A

지역 01 · 조회/저장

실행 레인 B

지역 02 · 조회/저장

실행 레인 ...

그룹 안의 나머지 지역

현재 그룹의 모든 작업 반환 대기



다음 그룹 시작

그룹 2 · 지역 21-40

이후 그룹도 같은 방식으로 반복

수동 업무와 자동 수집 구조를 비교한 개념도입니다. 지역 번호와 실행 레인은 설명용이며, 처리 시간이나 고정 스레드 수를 나타내지 않습니다.

구현 결과

담당자의 수동 수집 업무를 스케줄 실행·지역별 조회·응답 검증·전월 기준 회원수 저장으로 연결된 자동화 흐름으로 전환했습니다. 200개 이상 지역을 20개 단위로 나눠, 전체 호출을 한꺼번에 몰지 않으면서 여러 지역을 병렬 처리하도록 구현했습니다.

그룹·스레드·트랜잭션의 경계

구현 쟁점	선택과 동작
작업 그룹	최대 20개 지역씩 처리. 현재 그룹의 모든 작업이 반환된 뒤 다음 그룹 실행.
실행 스레드	parallelStream의 공용 ForkJoinPool과 호출 스레드 활용. 20은 고정 스레드 수가 아닌 그룹 크기.
요청 데이터	프레임 객체와 조회·저장용 Map을 작업별로 분리해 공유 가변 데이터 사용을 피함.
대기·예외	연결·읽기 타임아웃 각각 5초. 지역 작업의 예외를 기록하고 다른 지역 처리 계속. 전체 5초 제한과는 구분.

실행 그룹과 DB 트랜잭션은 서로 다른 단위

기존 Spring REQUIRED 트랜잭션 설정은 지역별 수집 서비스 호출에 적용됩니다. 각 호출에서 대표도서관을 찾고, 원격 회원수를 조회한 뒤 도서관별 결과를 월별 통계로 저장합니다. 20개 지역 그룹 전체를 하나의 트랜잭션으로 묶지 않습니다.

지역 01 트랜잭션 01 · REQUIRED	대표도서관 확인	→	원격 회원수 조회	→	월별 통계 INSERT
지역 02 트랜잭션 02 · REQUIRED	대표도서관 확인	→	원격 회원수 조회	→	월별 통계 INSERT

그룹 = 실행을 나누는 단위 / 트랜잭션 = 지역 서비스의 DB 처리 단위

지역 서비스 밖으로 전파되는 예외에는 rollback-for=Exception 규칙이 적용됩니다. 오류 프레임 반환은 예외 발생과 구분하며, 원격 도서관 조회와 센터 저장을 하나의 분산 트랜잭션으로 처리하지 않습니다.

그룹 간 순차 실행 · 그룹 내 병렬 실행

```
for (List<String> group : partition(regionCodes, 20)) {
    group.parallelStream().forEach(region -> {
        try {
            // Separate request data for each region.
            Frame request = createRequest(region);
            // Call the existing transactional service proxy.
            statisticsService.collectAndStore(request);
        } catch (Exception e) {
            logFailure(region, e);
        }
    }); // Wait for this group before starting the next.
}
```

핵심 로직 예시 · 서비스명과 업무 식별자는 변경했습니다.

회원 상태를 반영한 데이터 일관성 보정

㈜이씨오 | 책임음 연계 서비스 · 2025.03-06

담당 범위 · 지역별 오류 추적 · 회원 상태별 대출정지일 보정 · 유효성 검증

회원 상태와 날짜 누락 여부에 따라 대출정지일 보정 규칙을 나누어, 도서관과 통합센터 사이의 데이터 불일치를 개선했습니다.

문제와 접근

지역별 동기화 과정에서 회원 상태와 대출정지일이 일치하지 않는 문제가 발생했습니다. 날짜가 비어 있다는 이유만으로 센터 값을 덮어쓰면 유효한 정지 정보까지 지워질 수 있었습니다.

지역별 처리 흐름과 누락된 조건을 추적했습니다. 정상 회원은 정지일을 초기화하고, 정지 회원의 날짜가 누락되면 기존 센터 값을 유지하도록 수정했습니다. 날짜 단위로 값을 비교해 차이가 있을 때 갱신했습니다.

구현 흐름

도서관 상태·날짜 수신 → 정상·정지·누락 구분 → 센터 값과 날짜 비교 → 차이가 있을 때 보정

설계 판단

쟁점	구현한 선택	의미와 고려사항
정상 회원	보정 후보의 대출정지일을 빈 값으로 변경	회원 상태와 남아 있는 정지일의 불일치를 정리
정지 + 정지일 누락	기존 센터 대출정지일을 유지	누락된 로컬 값만으로 센터의 유효한 값을 지우지 않음
정지 + 정지일 존재	로컬의 정지일을 비교 후보로 사용	업무 상태에 맞는 값을 기준으로 보정
시각만 다름	일자 부분을 비교해 갱신 여부 결정	해당 필드의 업무 비교 단위를 날짜로 한정

결과

정상 회원에게 남아 있던 정지일을 정리하고, 날짜가 누락된 정지 회원은 기존 센터 값을 유지하도록 보정했습니다. 상태에 따른 갱신 기준을 적용해 잘못된 초기화를 방지했습니다.

기술 · Java, 업무 규칙, 데이터 정합성, 예외 처리

상태와 누락 여부에 따른 보정 후보 결정

정상 회원은 정지일을 비우고, 정지 회원의 날짜가 누락된 경우에는 기존 값을 보존합니다.

정상 초기화 / 정지일 누락 시 기존 값 유지

```
String candidate = localStopDate;
if ("0".equals(memberState)) {
    candidate = "";
} else if ("1".equals(memberState)
    && "".equals(localStopDate)) {
    candidate = centerStopDate;
}
// Compare the date portion before updating.
```

- 0은 정상, 1은 대출정지 상태이며 누락된 날짜는 빈 문자열로 처리합니다.
- 보정 후보와 센터 값의 날짜 부분을 비교한 뒤 필요한 경우 갱신합니다.

핵심 로직 예시 · 업무 식별자는 변경했습니다.

연관 작업 · WEB 탈퇴 경로의 처리 방식 통일 (2025.03)

대부분의 탈퇴 서비스에는 soft delete가 적용돼 있었지만, WEB 탈퇴 경로에는 물리 삭제가 남아 있었습니다. 팀에서 합의한 soft delete 정책에 맞춰 해당 경로를 상태 변경 방식으로 수정했습니다. 탈퇴 지역의 회원 상태를 갱신하고, 탈퇴하지 않은 가입지역이 더 이상 없을 때 대상 개인정보 필드를 NULL로 정리하도록 구현해 경로별 처리 방식의 차이를 해소했습니다.

운영 조회 도구 개선과 이기종 연계

㈜이씨오 | 책임음 운영 도구 · 2026.05-06

담당 범위 · 이기종 조회 분기·응답 변환 개발 및 운영 반영 · 주말민원 통합 조회 API·SQL·화면 개선

외부 업체·사서를 거치거나 수동 쿼리 요청이 필요했던 이기종 도서관 이용자 조회를 운영 화면에 구현하고 운영에 반영했습니다. 주말민원 도구는 가입지역 상태·연체자료까지 함께 확인하도록 확장했습니다.

문제와 접근

이기종 도서관은 운영 화면에서 직접 조회할 수 없어 유지보수업체·사서에게 확인하거나 IL_K95 서비스로 쿼리를 수동 요청해야 했습니다. 주말민원 도구도 기본정보만 제공해 지역별 상태와 연체자료를 함께 파악하기 어려웠습니다.

도서관 설정에 따라 기존 조회 경로와 IL_K95 기반 조회 경로를 선택하고, 시스템마다 다른 응답을 공통 이용자 정보로 변환했습니다. 별도의 주말민원 화면에는 가입지역·연체자료 조회와 동기화 완료 후 재조회 흐름을 추가했습니다.

구현 흐름

도서관별 조회 설정 확인 → 기존·이기종 경로 선택 → 시스템별 응답 변환 → 공통 이용자 정보 표시

설계 판단

쟁점	구현한 선택	의미와 고려사항
조회 경로 분리	도서관 설정과 조회 쿼리 유무에 따라 기존 경로 또는 IL_K95 호출	기존 조회를 유지하면서 이기종 도서관으로 조회 범위 확장
응답 구조 통일	ILUS·SOLAS·K3 계열 응답을 공통 이용자 항목으로 매핑	도서관 시스템 차이를 운영 화면에서 일관되게 표시
응답 검증	오류 응답·파싱 실패·이용자 정보 누락 확인	비정상 응답을 정상 조회 결과와 구분
주말민원 화면 확장	기본정보·가입지역·연체자료 통합 조회, 동기화 완료 후 재조회	민원 판단과 조치 후 확인을 한 화면에서 수행

결과

직접 개발한 이기종 조회 기능을 운영에 반영해 외부 담당자 확인이나 수동 쿼리 요청을 거치던 조회를 운영 화면에서 수행할 수 있게 했습니다. 주말민원 담당자는 한 화면에서 지역별 상태·연체자료를 확인하고 동기화 후 결과를 다시 조회할 수 있게 됐습니다.

기술 · Spring MVC, MyBatis, JSP · AJAX, 운영 도구

도서관별 조회 경로 선택과 응답 통일

도서관별 설정으로 조회 경로를 결정합니다. 이기종 응답은 시스템 유형에 맞춰 해석한 뒤, 운영 화면에서 사용하는 공통 이용자 항목으로 변환합니다.

설정 확인 → 조회 경로 분기 → 공통 응답 변환

```
LibraryConfig config = findLibraryConfig(regionCode);
if (!config.isStandard()
    && hasQuery(config.getQuery())) {
    Frame response = requestK95(config, userNo, ci);
    validateResponse(response);
    return mapToCommonUser(response, config.getType());
}
```

```
Frame response = requestK117(libraryCode, userNo);
validateResponse(response);
return readStandardUser(response);
```

- 기존 조회 경로를 유지하고, 별도 조회 설정이 있는 이기종 도서관에 분기 처리를 적용했습니다.
- 시스템별 응답 항목을 이용자 번호·회원 상태·대출정지일 등 공통 필드에 매핑했습니다.
- 프레임 오류와 파싱 실패, 이용자 정보 누락을 정상 결과와 구분해 처리했습니다.

핵심 로직 예시 · 업무 식별자는 변경했습니다.

주말민원 화면의 확장

기능 비교용 재구성 화면입니다. 개인정보 보호를 위해 가상 데이터를 사용했습니다.

기능 비교용 재구성 화면 · 가상 데이터

기존 기본정보 조회 구성

userinfo1	userinfo1	userinfo1
DEMO-0001	예시 이용자	내부 식별값 생략

책이름 이용자 동기화

책이름 이용자 번호 입력

이용자 조회

동기화 실행

이용 방법

1. 이용자 번호를 입력하여 이용자 조회를 합니다.
2. 조회된 가입지역과 연체자료를 확인합니다.
3. 동기화 요청 후 상태를 다시 확인합니다.

기존 구성 · 기본정보 중심

확장 화면 · 조회 전

확장 화면 · 조회 후

기능 비교용 재구성 화면 · 가상 데이터

책이름 이용자 동기화

DEMO-0001 (가상 이용자)

이용자 조회

동기화 실행

이용자 정보

대출자번호	DEMO-0001	이름	예시 이용자	연락처	연락처 비공개
-------	-----------	----	--------	-----	---------

가입지역 상태

No	가입지역	회원상태	대출증	대출정지일
1	예시 지역 A	정상	0	-
2	예시 지역 B	정지	0	2026-10-15
3	예시 지역 C	정상	0	-

연체자료 목록

No	구분	도서관	서명	반납예정일	연체일수
연체자료가 없습니다.					

이용 방법

1. 이용자 번호를 입력하여 이용자 조회를 합니다.
2. 조회된 가입지역과 연체자료를 확인합니다.
3. 동기화 요청 후 상태를 다시 확인합니다.

기본정보·가입지역 상태·연체목록을 함께 보여줍니다.